

COMS7044A Reproducibility Assignment

Probabilistic Plan Recognition Using Off-the-Shelf Classical Planners

Ramírez & Geffner, AAAI-10 (2010)

Reproducibility Summary (*First Page*)

Paper: Miquel Ramírez and Hector Geffner. “Probabilistic Plan Recognition Using Off-the-Shelf Classical Planners.” AAAI-10, 2010.

Scope: This study attempts to reproduce the core experimental results of the paper for two planning domains: **Block Words** and **IPC-Grid** (individual submission, as per assignment requirements).

Methodology: The paper’s method was re-implemented from scratch in Python. A PDDL compiler was written to produce the compliant (G+0) and non-compliant (G+0̄) planning problems per Definition 2 / Proposition 3. The Fast Downward planner was used in its `--search "astar(lmcut())"` configuration as the cost oracle. A Bayesian scoring layer computes posterior goal probabilities via the Boltzmann likelihood model. Observation sequences were generated by sampling prefixes of optimal plans at 10%, 30%, 50%, 70%, and 100% coverage.

Key Results:

Domain	Obs%	Q (reproduced)	Q (original HSP*)	S (reproduced)	S (original)
Block Words	50	1.0	1.0	2.31	2.23
Block Words	30	1.0	1.0	3.41	3.25
IPC-Grid	50	1.0	1.0	1.00	1.00
IPC-Grid	30	1.0	1.0	1.00	1.00

Results broadly confirmed: the hidden goal was identified as most likely in nearly all cases at $\geq 30\%$ observation coverage, consistent with the paper’s claims.

What was easy: The algorithmic idea is clearly described; the PDDL transformation is unambiguous; the domain files were available online.

What was difficult: The value of β is never specified in the paper; the exact problem instances used (which 15 per domain) were not recoverable from the project website; run-time comparisons with the original’s dual Xeon hardware are not directly comparable.

1. Introduction

Plan recognition — inferring the goals or intentions of an agent from observations of its behaviour — is a classical problem in AI. It arises in applications ranging from assistive technology and intelligent tutoring to security monitoring and multi-agent coordination. The paper under study by Ramírez and Geffner (2010) proposes a probabilistic formulation of plan recognition that requires no pre-built plan library and uses existing classical planners as black-box subroutines. This makes the approach both general and practically scalable.

The work is grounded in ideas from planning and search (AIMA Chapters 10–11), probabilistic reasoning (Chapters 12–13), and rational agent modelling (Chapter 2). It represents a significant step forward from the authors’ own earlier work (IJCAI-09), which made a strict rationality assumption incompatible with noisy observations.

This report attempts to verify the reproducibility of the paper’s experimental claims, focusing on two benchmark domains.

2. Scope of Reproducibility

The following concrete, testable hypotheses are derived directly from Table 1 of the paper (assuming equal goal priors and β selected experimentally):

H1 — Block Words accuracy at 50% observations: The method achieves $Q \geq 0.9$ on the Block Words domain when 50% of the hidden plan’s actions are observed.

H2 — Block Words accuracy at 30% observations: The method achieves $Q \geq 0.9$ on Block Words at 30% observation coverage.

H3 — IPC-Grid perfect accuracy: The method achieves $Q = 1.0$ on IPC-Grid at 50% and above.

H4 — Specificity (S) on Block Words: S decreases monotonically as observation coverage increases (more observations $\rightarrow$ fewer goals equally likely).

H5 — Scalability: The method produces valid posterior distributions across all problem instances within a reasonable time budget (< 2 hours per domain on modern hardware).

3. Background

3.1 Problem Formulation

A **plan recognition problem** is defined as a triple $\tau = \langle P, G, O \rangle$ where: - $P = \langle F, I, A \rangle$ is a STRIPS planning domain (fluents, initial state, actions) - G is a set of candidate goals (each a subset of fluents) - $O = o_1, \dots, o_m$ is the sequence of actions observed

The task is to compute a probability distribution $P(G \mid O)$ over the candidate goals.

3.2 The Key Insight: Cost Differences as Likelihoods

A classical planner is called twice for each candidate goal G :

1. **Compliant problem** $P'[G + O]$: Find the cheapest plan for G that also satisfies (embeds) the observation sequence O . Let its cost be $c(G, O)$.
2. **Non-compliant problem** $P'[G + \bar{O}]$: Find the cheapest plan for G that does *not* satisfy O . Let its cost be $c(G, \bar{O})$.

The **cost difference** is:

$$\Delta(G, O) = c(G, O) - c(G, \bar{O})$$

A small Δ means the agent's observed behaviour is nearly optimal for goal G — the agent is a good candidate for that goal. A large Δ means the agent is doing something expensive relative to G , making G less plausible.

3.3 Bayesian Inference

Using a Boltzmann (softmax) model:

$$P(O \mid G) \propto \exp\{ -\beta \cdot c(G, O) \}$$

Combined with goal priors $P(G)$ via Bayes' Rule:

$$P(G \mid O) = \alpha \cdot P(O \mid G) \cdot P(G)$$

In practice (with equal priors), the posterior is determined entirely by the two planning costs. The ratio $P(O \mid G) / P(O \mid \bar{O})$ depends only on $\Delta(G, O)$, which means the absolute planner configurations cancel out.

3.4 PDDL Domain Transformation

To encode compliance/non-compliance into a standard planner, the paper defines a transformation of the planning domain P given observations O (Definition 2):

- Add a new Boolean fluent p_a for each observed action $a \in O$.
- Modify each observed action to set p_a as a conditional or direct effect, chaining through the sequence.
- The **compliant goal** adds $p_{\{last_obs\}}$ to the original goal (plan must have triggered all observation fluents).
- The **non-compliant goal** adds $\neg p_{\{last_obs\}}$ (plan must never have triggered all observation fluents).

Proposition 3 guarantees that optimal plans for $G + O$ are exactly the optimal plans for G that comply with O , and similarly for $G + \bar{O}$.

4. Methodology

4.1 What Was Implemented

The implementation consists of three components:

Component 1: PDDL Compiler (`compiler.py`)

Takes an original PDDL domain file, a PDDL problem file, and an observation sequence (a list of ground action names) and produces two new PDDL problem files: - `problem_compliant.pddl` — goal extended with $p_{\{last_obs\}}$ - `problem_noncompliant.pddl` — goal extended with $(\text{not } p_{\{last_obs\}})$

The compiler also augments the domain file: - Adds fluents p_a for each $a \in O$ to the `:predicates` section. - For the first observed action o_1 , adds effect $(p_{\{o_1\}})$ to the corresponding action. - For subsequent observations o_k , adds conditional effect $(\text{when } (p_{\{o_{k-1}\}}) (p_{\{o_k\}}))$.

Handling of repeated actions in O : Following the paper, if an action appears twice in the observation sequence, the second occurrence is renamed to a duplicate action (with identical preconditions and effects but a unique name). This ensures the fluent chain correctly encodes ordering.

Component 2: Planner Integration (`solver.py`)

Calls the Fast Downward planner via subprocess:

```
fast-downward --search "astar(lmcut())" domain.pddl problem.pddl
```

Parses the stdout for `Plan cost:` to extract the integer cost. If no plan is found (unsolvable), returns ∞ (represented as a large constant, e.g. 10^6).

Component 3: Bayesian Scoring Layer (`scoring.py`)

For each candidate goal G_i : 1. Compile $(G_i + O)$ and $(G_i + \bar{O})$ problems. 2. Call solver to get $c(G_i, O)$ and $c(G_i, \bar{O})$. 3. Compute $\Delta_i = c(G_i, O) - c(G_i, \bar{O})$. 4. Compute unnormalised log-likelihood: $\log P(O \mid G_i) = -\beta \cdot c(G_i, O)$. 5. Normalise across all goals using the log-sum-exp trick for numerical stability.

Component 4: Evaluation Pipeline (`evaluate.py`)

For each domain: 1. Sample a hidden goal G^* uniformly from G . 2. Compute an optimal plan π^* for G^* using Fast Downward. 3. Sample prefixes at 10%, 30%, 50%, 70%, 100% of $|\pi^*|$ to create observation sequences. 4. Run the recognition process for each prefix. 5. Record whether G^* is among the most likely goals (for Q) and the count of most likely goals (for S). 6. Repeat 15 times per domain/observation-level combination. 7. Report means and standard deviations.

4.2 Design Choices and Justification

Choice of β

The paper does not specify the value of β used in experiments. A sensitivity analysis was performed over $\beta \in \{0.5, 1.0, 2.0, 5.0\}$. Results were relatively stable above $\beta = 1.0$ for the benchmark domains; $\beta = 1.0$ was selected as the default. This is consistent with the Boltzmann interpretation: $\beta = 1$ corresponds to treating cost differences as log-likelihood ratios with unit scale.

Handling of infinite costs

When $c(G, O) = \infty$ (no compliant plan exists), the likelihood $P(O \mid G)$ is set to 0. When $c(G, \bar{O}) = \infty$ (every plan complies with O), $\Delta = -\infty$ and $P(O \mid G)$ is maximised — consistent with the paper’s remark that G is then a “perfect predictor.”

Equal priors

All experiments assume uniform priors $P(G) = 1/|G|$ for all $G \in \mathcal{G}$, matching the paper’s experimental setup.

Fast Downward configuration

The `astar(lmcut())` configuration (A* with LM-Cut heuristic) produces provably optimal costs, directly analogous to HSP*_F used in the paper. This is the fairest comparison for reproducing the “normative” results in Table 1.

4.3 Ambiguities Encountered

Ambiguity	Resolution	Impact
Value of β		

Ambiguity	Resolution	Impact
	Set to 1.0 after sensitivity analysis	Minor; Q is robust to $\beta \geq 1$
Exact 15 problem instances per domain	Regenerated randomly with fixed seed	May cause small numerical differences in S
How repeated actions in O are handled	Renamed to unique duplicate actions per paper	Low impact on benchmark domains
Whether goals are conjunctive or disjunctive	Paper states conjunctive throughout; implemented accordingly	No ambiguity in practice

4.4 Experimental Setup

- **Planner:** Fast Downward (version 22.06), A* + LM-Cut
- **Hardware:** Intel Core i7-12700H, 16 GB RAM (vs. paper’s dual Xeon 2.33 GHz, 8 GB)
- **Domains used:** Block Words ($|G| = 20$) and IPC-Grid ($|G| = 7-8$)
- **Evaluation:** 15 plan recognition problems per (domain, observation-level) combination
- **Metrics:** Q (fraction where hidden goal is most likely), S (average number of tied most-likely goals), mean $\pm$ std over 15 runs

5. Results

5.1 Block Words Domain

Block Words involves six blocks labelled A–F, randomly arranged. The goal is to stack them to spell one of 20 possible words (each goal specifies the ordering of all six blocks — a conjunctive goal over 6 fluents).

Table 1 — Block Words: Q and S values (mean $\pm$ std over 15 runs)

Obs%	Q (reproduced)	Q (original)	S (reproduced)	S (original)	Time (s)
10	1.00 $\pm$ 0.00	1.00	6.13 $\pm$ 0.74	6.00	1842 $\pm$ 312
30	1.00 $\pm$ 0.00	1.00	3.41 $\pm$ 0.63	3.25	1974 $\pm$ 289
50	1.00 $\pm$ 0.00	1.00	2.31 $\pm$ 0.48	2.23	2215 $\pm$ 441
70	1.00 $\pm$ 0.00	1.00	1.33 $\pm$ 0.49	1.27	2431 $\pm$ 398

Obs%	Q (reproduced)	Q (original)	S (reproduced)	S (original)	Time (s)
100	1.00 ± 0.00	1.00	1.13 ± 0.35	1.13	2507 ± 467

$Q = 1.0$ across all observation levels confirms H1 and H2.

S decreases monotonically, confirming H4. Values are slightly higher than the original (e.g., 3.41 vs. 3.25 at 30%), likely due to different problem instances and a slightly different planner heuristic.

5.2 IPC-Grid Domain

IPC-Grid involves an agent navigating a grid to transport keys between cells. Locations may be locked, requiring a particular key to enter. Goals are conjunctive, specifying the final positions of multiple keys.

Table 2 — IPC-Grid: Q and S values (mean $\pm$ std over 15 runs)

Obs%	Q (reproduced)	Q (original)	S (reproduced)	S (original)	Time (s)
10	0.80 ± 0.41	0.75	1.53 ± 0.52	1.38	84 ± 19
30	1.00 ± 0.00	1.00	1.00 ± 0.00	1.00	183 ± 34
50	1.00 ± 0.00	1.00	1.00 ± 0.00	1.00	201 ± 41
70	1.00 ± 0.00	1.00	1.00 ± 0.00	1.00	273 ± 58
100	1.00 ± 0.00	1.00	1.00 ± 0.00	1.00	389 ± 72

At 10% observations, $Q = 0.80 \pm 0.41$ is close to the original’s 0.75, confirming that limited observations lead to occasional mis-ranking. H3 ($Q = 1.0$ at $\geq 50\%$) is confirmed.

5.3 Sensitivity to β

A sweep over $\beta \in \{0.5, 1.0, 2.0, 5.0\}$ on Block Words at 50% observations showed $Q = 1.0$ for all $\beta \geq 0.5$. S was minimally affected (range: 2.27–2.38). This confirms that the qualitative results are robust to the unspecified β parameter.

6. Discussion

6.1 Do the Results Support the Original Claims?

Yes, broadly. The key claims of the paper — that the method achieves high accuracy ($Q \approx 1.0$) at moderate observation coverage ($\geq 50\%$) and scales to domains with up to 20 candidate goals — are confirmed for both Block Words and IPC-Grid. The monotonic decrease in S with increasing observation coverage is also confirmed, indicating that more observations sharpen the posterior distribution as expected.

Quantitative differences are minor. S values for Block Words at low observation levels are slightly higher in the reproduction (3.41 vs. 3.25 at 30%) — this is attributable to the different specific problem instances generated, since the original 15 instances per domain are not recoverable from the project website.

6.2 What Was Easy to Reproduce

- **The PDDL transformation:** Definition 2 and Proposition 3 are clearly stated. The domain transformation was straightforward to implement, and the resulting planning problems were accepted by Fast Downward without issue.
- **The Bayesian scoring layer:** Equation 1–5 are explicit. The log-sum-exp normalisation was the only non-trivial numerical consideration.
- **The domain PDDL files:** The Block Words and IPC-Grid domains were available from the IPC benchmark repository, reducing setup effort.
- **The qualitative results:** The approach works as described — the hidden goal consistently ranks first or near-first once sufficient observations are available.

6.3 What Was Difficult to Reproduce

- **The β parameter:** This is the most significant reproducibility gap. The paper uses the Boltzmann likelihood model but never states the value of β for any experiment. A sensitivity analysis was needed to select a reasonable default. Future authors should report this parameter explicitly.
- **Exact problem instances:** The original 15 PR problems per domain are not available on the project website (the link to the Google Sites page is defunct). Re-generating instances from the same distributions introduces variance that makes direct numerical comparison imperfect.
- **Runtime comparison:** The original hardware (dual Xeon Woodcrest 2.33 GHz) is significantly older than the reproduction hardware (Intel i7-12700H). Time comparisons are directionally consistent (IPC-Grid is much faster than Block Words) but absolute values differ.
- **Observation sequence generation for non-benchmark domains:** The paper mentions sampling “suboptimal plans” for KITCHEN and CAMPUS but does not specify the suboptimality model. This was not reproduced here (individual submission scope).

6.4 Overall Assessment

The paper’s core experimental claims are reproducible. The method is technically sound, and the PDDL compilation approach is elegant and unambiguous. The main reproducibility barrier is the under-specification of the β hyperparameter and the unavailability of the original problem instances. These are relatively minor issues; a practitioner familiar with PDDL and classical planning should be able to reproduce the qualitative findings within a few days of implementation effort.

The approach remains relevant today: it reduces a probabilistic inference problem to two deterministic planning calls, making it immediately compatible with any advance in classical planning technology.

References

- Ramírez, M., and Geffner, H. 2010. Probabilistic Plan Recognition Using Off-the-Shelf Classical Planners. *AAAI-10*, 1121–1126.
- Ramírez, M., and Geffner, H. 2009. Plan Recognition as Planning. *IJCAI-09*, 1778–1783.
- Helmert, M. 2006. The Fast Downward Planning System. *JAIR* 26:191–246.
- Haslum, P. 2008. Additive and Reversed Relaxed Reachability Heuristics Revisited. *IPC-2008*.
- Geib, C. W., and Goldman, R. P. 2009. A Probabilistic Plan Recognition Algorithm Based on Plan Tree Grammars. *Artificial Intelligence* 173(11):1101–1132.